



Wrocław
University
of Science
and Technology

Software Engineering

API and contract design

Marcin Jodłowiec

December, 17th, 2025

Agenda

- Introduction & Motivation
- Introduction
- Explicit contract modeling
- Design-First Approach
- API Styles and Contracts
- Contract Testing and Evolution
- Conclusion

Introduction & Motivation

Introduction

What Is an API and Who Needs It?

- ▶ An **API (Application Programming Interface)** is a formally defined interface that allows software systems to communicate.
- ▶ An API specifies:
 - ▶ which operations can be invoked,
 - ▶ what data is exchanged,
 - ▶ how errors are reported,
 - ▶ what behavior is guaranteed.
- ▶ APIs are needed by:
 - ▶ **client applications** consuming functionality,
 - ▶ **backend services** exposing capabilities,
 - ▶ **independent teams** integrating their systems,
 - ▶ **external partners** accessing shared platforms.
- ▶ APIs enable **decoupling, independent evolution, and scalable system integration.**

Types of APIs

- ▶ The term **API** is used broadly in programming and software engineering to describe different kinds of interfaces.
- ▶ Common types of APIs include:
 - ▶ **Library APIs**: function and class interfaces exposed by libraries or frameworks (e.g. standard libraries, SDKs).
 - ▶ **Operating system APIs**: interfaces for interacting with the OS (e.g. file systems, processes, networking).
 - ▶ **Hardware APIs**: low-level interfaces to devices and peripherals.
 - ▶ **Service APIs**: network-accessible interfaces exposed by services or systems.
- ▶ This lecture tells more on **service APIs**, in particular:
 - ▶ REST APIs,
 - ▶ GraphQL APIs,
 - ▶ RPC-based APIs (e.g. gRPC),
 - ▶ asynchronous and event-driven APIs.

Problems with APIs

- ▶ API failures are rarely caused by algorithms or performance issues.
- ▶ Common failure modes include:
 - ▶ mismatched expectations between API producers and consumers,
 - ▶ undocumented assumptions about data or behavior,
 - ▶ breaking changes introduced without coordination,
 - ▶ uncontrolled or inconsistent versioning.
- ▶ These failures typically arise when system behavior is implicit rather than explicitly specified.

APIs as Contracts

- ▶ An API defines an agreement between a service and its consumers.
- ▶ The agreement specifies:
 - ▶ available operations,
 - ▶ exchanged data,
 - ▶ error conditions,
 - ▶ expected behavior.
- ▶ Consumers build logic based on this agreement.
- ▶ Violating the agreement causes integration and runtime failures.

Explicit contract modeling

What Is an API Contract?

Definition

An API contract is a formal, machine-readable specification that defines the interface and observable behavior of a service.

- ▶ operations and their signatures,
- ▶ input and output data models,
- ▶ error semantics,
- ▶ behavioral guarantees.

Contract Elements

- ▶ **Operations:** names, parameters, return values.
- ▶ **Data models:** schemas, types, constraints, multiplicities.
- ▶ **Errors:** status codes, error payloads, semantic meaning.
- ▶ **Behavior:** ordering, idempotency, side effects, consistency guarantees.

Modeling Exchanges

- ▶ APIs model exchanges between systems, not just endpoints.
- ▶ Exchange characteristics include:
 - ▶ request-response vs event-based interaction,
 - ▶ payload structure and size,
 - ▶ frequency and throughput,
 - ▶ synchronous vs asynchronous execution.
- ▶ Exchange modeling directly influences performance and scalability.

Contract vs Documentation

- ▶ Documentation is primarily human-readable.
- ▶ Contracts are machine-readable and enforceable.
- ▶ Documentation may become outdated.
- ▶ Contracts can be:
 - ▶ validated automatically,
 - ▶ tested in CI/CD pipelines,
 - ▶ used to generate mocks and clients.

Design-First Approach

Design-First vs Code-First

	Design-First	Code-First
Starting point	API contract	Implementation
Source of truth	Specification	Code
Documentation	Derived from contract	Generated post-hoc
Testing	Contract-driven	Implementation-driven
Change visibility	Explicit	Often implicit

Design-First Lifecycle

- ▶ design the API contract,
- ▶ review the contract with stakeholders,
- ▶ validate behavior using mocks,
- ▶ implement the service,
- ▶ verify implementation against the contract,
- ▶ evolve the contract in a controlled manner.

API Styles and Contracts

API Styles Overview

- ▶ **REST APIs with OpenAPI specifications,**
- ▶ GraphQL APIs with schema definitions,
- ▶ RPC APIs using gRPC and Protocol Buffers,
- ▶ asynchronous APIs specified using AsyncAPI.

REST and OpenAPI

- ▶ resource-oriented interaction model,
- ▶ HTTP methods and status codes,
- ▶ OpenAPI as a formal contract specification.

```
1 paths:
2   /users/{id}:
3     get:
4       responses:
5         "200":
6           description: User representation
7           content:
8             application/json:
9               schema:
10                 $ref:
11                   "#/components/schemas/User"
12
13 components:
14   schemas:
15     User:
16       type: object
17       required: [id, name, email]
```

what is this famous REST?

- ▶ **REST (Representational State Transfer)** is an architectural style defined by a set of constraints for network-based systems.
- ▶ REST is most commonly implemented using HTTP as the transport protocol, but it is independent of any specific protocol.
- ▶ According to Roy Fielding, a RESTful system satisfies the following constraints:
 - ▶ **Resource-oriented interaction:** the producer exposes resources that consumers interact with.
 - ▶ **Stateless communication:** each request contains all information required for processing; no client session state is stored on the server.
 - ▶ **Cacheability:** responses explicitly define whether they can be cached.
 - ▶ **Uniform interface:** consistent use of identifiers, representations, and operations.
 - ▶ **Layered system:** clients are unaware of intermediate components or internal structure.

HTTP Methods and Status Codes

HTTP Method	Typical Use	Common Status Codes
GET	Retrieve a resource representation	200 OK, 304 Not Modified, 404 Not Found
POST	Create a new resource or trigger processing	201 Created, 202 Accepted, 400 Bad Request
PUT	Create or fully replace a resource	200 OK, 204 No Content, 404 Not Found
PATCH	Partially update a resource	200 OK, 204 No Content, 400 Bad Request
DELETE	Remove a resource	200 OK, 204 No Content, 404 Not Found
HEAD	Retrieve metadata without body	200 OK, 404 Not Found
OPTIONS	Discover supported methods and capabilities	200 OK

Status codes are part of the API contract – they indicate the outcome of a request.

Richardson Maturity Model

- ▶ The **Richardson Maturity Model (RMM)** classifies HTTP-based APIs according to how closely they follow REST principles.
- ▶ **Level 0 – HTTP as RPC:**
 - ▶ a single endpoint,
 - ▶ HTTP used as a transport protocol,
 - ▶ intent encoded in the payload.
- ▶ **Level 1 – Resources:**
 - ▶ multiple resource-oriented URLs,
 - ▶ basic identification of domain objects,
 - ▶ limited use of HTTP semantics.
- ▶ **Level 2 – HTTP Methods:**
 - ▶ correct use of HTTP verbs (GET, POST, PUT, DELETE),
 - ▶ clear separation of read and write operations,
 - ▶ reliance on HTTP status codes and semantics.
- ▶ **Level 3 – Hypermedia Controls (HATEOAS):**
 - ▶ responses include links describing possible next actions,
 - ▶ clients navigate the API dynamically,
 - ▶ minimal coupling to predefined interaction flows.

Practical Applications of OpenAPI Specifications I

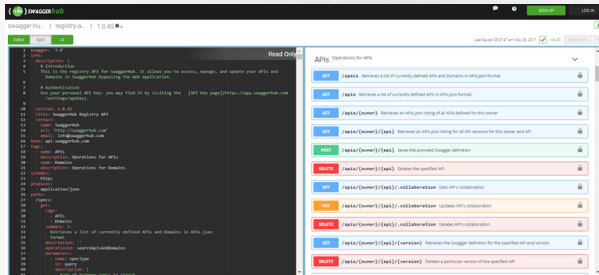
- ▶ An **OpenAPI Specification (OAS)** is not only documentation, but a machine-readable artifact enabling multiple engineering activities.
- ▶ **Code generation:**
 - ▶ generation of client libraries and server stubs,
 - ▶ support for multiple languages and frameworks,
 - ▶ consistent API usage across teams.
- ▶ **Validation:**
 - ▶ validation of requests and responses against the specification,
 - ▶ detection of malformed or non-conformant exchanges,
 - ▶ enforcement of API contracts at runtime or in CI/CD.
- ▶ **Examples and mocking:**
 - ▶ example requests and responses embedded in the specification,
 - ▶ mock servers generated directly from the OAS,

Practical Applications of OpenAPI Specifications II

- ▶ early feedback for consumers before implementation is complete.
- ▶ **Change detection:**
 - ▶ identification of backward-incompatible changes,
 - ▶ support for controlled API evolution and versioning.

Swagger and OpenAPI

- **Swagger** was the original name for a set of tools and specifications for designing, building, documenting, and consuming RESTful APIs.



OpenAPI Tooling and Ecosystem

- ▶ OpenAPI specifications are supported by a broad ecosystem of tools covering the entire API lifecycle.
- ▶ **Specification-driven tools** (specification → code):
 - ▶ **openapi-generator**: client libraries and server stubs for multiple languages and frameworks,
 - ▶ **Swagger Codegen**: legacy code generation tooling,
 - ▶ generation of SDKs, mocks, and documentation from OAS.
- ▶ **Implementation-driven tools** (code → specification):
 - ▶ frameworks that generate OpenAPI from code annotations or type hints,
 - ▶ examples: **FastAPI**, Springdoc (Spring Boot), ASP.NET Web API.
- ▶ Both approaches rely on OpenAPI as a **shared contract artifact** between producers, consumers, and tooling.

GraphQL Schema-First

- ▶ schema defined using Schema Definition Language (SDL),
- ▶ strong typing of queries and responses,
- ▶ schema as the primary contract artifact.

```
1 type User {  
2   id: ID!  
3   name: String  
4 }
```

- ▶ <https://graphql.org/learn/introduction/>

gRPC and RPC APIs

- ▶ interface definition using Protocol Buffers,
- ▶ service and method definitions in .proto files,
- ▶ binary payloads transported over HTTP/2.

```
service UserService {  
  rpc GetUser (UserId) returns (User);  
}
```

Asynchronous APIs

- ▶ event-driven communication model,
- ▶ loose coupling between producers and consumers,
- ▶ contracts defined using AsyncAPI specifications.

Performance and Trade-offs I

- ▶ **HTTP/1.1 vs HTTP/2**
 - ▶ HTTP/1.1: sequential request–response, head-of-line blocking at connection level,
 - ▶ HTTP/2: multiplexing multiple streams over a single TCP connection,
 - ▶ HPACK header compression reduces overhead for metadata-heavy APIs,
 - ▶ improved latency for many small requests, especially in high-latency networks.
- ▶ **Text-based vs binary serialization**
 - ▶ text formats (JSON, XML, GraphQL): human-readable, debuggable, larger payloads,
 - ▶ binary formats (Protobuf, FlatBuffers): compact, faster parsing, schema-driven,
 - ▶ binary formats reduce CPU usage and network transfer at the cost of tooling complexity.
- ▶ **Payload size and structure**

Performance and Trade-offs II

- ▶ larger payloads increase transmission time and memory pressure,
 - ▶ deeply nested structures increase parsing and deserialization cost,
 - ▶ overfetching and underfetching directly affect latency and bandwidth usage.
- ▶ **Protocol choice as an architectural decision**
 - ▶ protocol impacts scalability, observability, and failure modes,
 - ▶ affects caching strategies, load balancing, and backward compatibility,
 - ▶ should be aligned with system constraints (latency, throughput, evolution rate).

Protocol Comparison by Criteria I

- ▶ **Communication model**
 - ▶ REST: resource-oriented (CRUD over HTTP)
 - ▶ gRPC: RPC-oriented (service + method)
 - ▶ GraphQL: query-oriented (client-defined queries)
- ▶ **Transport**
 - ▶ REST: HTTP/1.1 or HTTP/2
 - ▶ gRPC: HTTP/2 (mandatory)
 - ▶ GraphQL: HTTP/1.1 or HTTP/2
- ▶ **Serialization format**
 - ▶ REST: text-based (JSON, XML)
 - ▶ gRPC: binary (Protocol Buffers)
 - ▶ GraphQL: text-based (JSON responses)
- ▶ **Payload control**
 - ▶ REST: fixed by endpoint
 - ▶ gRPC: fixed by method definition
 - ▶ GraphQL: client controls requested fields

Protocol Comparison by Criteria II

- ▶ **Over- and under-fetching**
 - ▶ REST: common problem
 - ▶ gRPC: possible, method-dependent
 - ▶ GraphQL: largely eliminated
- ▶ **Performance characteristics**
 - ▶ REST: good, predictable performance
 - ▶ gRPC: very high throughput, low latency
 - ▶ GraphQL: variable, query-dependent performance
- ▶ **Schema and contracts**
 - ▶ REST: informal contracts (OpenAPI)
 - ▶ gRPC: strong, mandatory schema (.proto)
 - ▶ GraphQL: strong, explicit schema
- ▶ **Caching**
 - ▶ REST: native HTTP caching
 - ▶ gRPC: limited, custom solutions
 - ▶ GraphQL: non-trivial, query-aware caching

Protocol Comparison by Criteria III

- ▶ **Error handling**
 - ▶ REST: HTTP status codes + response body
 - ▶ gRPC: structured status codes
 - ▶ GraphQL: partial errors in responses
- ▶ **Tooling maturity**
 - ▶ REST: very high
 - ▶ gRPC: high (strongly typed tooling)
 - ▶ GraphQL: high (query tooling, introspection)
- ▶ **Typical use cases**
 - ▶ REST: public APIs, CRUD-oriented services
 - ▶ gRPC: internal microservices, low-latency systems
 - ▶ GraphQL: frontend-driven data aggregation
- ▶ **Learning curve**
 - ▶ REST: low
 - ▶ gRPC: medium
 - ▶ GraphQL: medium to high

Contract Testing and Evolution

Contract Testing and API Evolution

- ▶ **Contract testing** verifies whether an API implementation conforms to the agreed contract, independently of internal logic.
- ▶ The focus is on **externally observable behavior**:
 - ▶ operation signatures,
 - ▶ response structures,
 - ▶ error semantics,
 - ▶ behavioral guarantees.
- ▶ Contract testing supports safe API evolution by distinguishing:
 - ▶ **non-breaking changes**: adding optional fields, adding new endpoints,
 - ▶ **breaking changes**: removing fields, changing semantics, tightening constraints.
- ▶ In practice, contract testing acts as a **compatibility gate** in CI/CD pipelines, preventing unintended breaking changes.

Conclusion

Engineering Principles for API Design

- ▶ APIs should be treated as long-lived interfaces,
- ▶ contracts should be explicit and validated,
- ▶ evolution must be planned and controlled,
- ▶ automation is required to maintain consistency.